

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO3: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Case Study

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Ashira Bridget Susan S

192524316

1. A multinational software company develops a large ERP application consisting of 120 dynamically linked libraries. During execution, users complain that startup time is high while memory utilization remains low. As the system architect, analyze whether static linking or dynamic linking is the better choice. Recommend an optimized loading strategy by considering compile time, load time, execution time, and shared libraries. Justify your decision with suitable memory diagrams.

Problem Analysis

A multinational software company develops an ERP application consisting of 120 dynamically linked libraries (DLLs). Users report:

- High startup time
- Low memory utilization

The architect must decide whether static linking or dynamic linking is better and recommend an optimized loading strategy.

Static Linking

Working

- All required libraries are combined with the executable during compilation.
- The executable contains all required code.

Advantages

- Faster execution because library lookup is unnecessary.
- No dependency on external libraries.
- Simple loading process.

Disadvantages

- Large executable size.
- Every process stores its own copy of library code.
- Higher memory consumption.
- Updating libraries requires recompilation.

Dynamic Linking

Working

- Libraries remain separate.
- They are loaded only when needed.
- Multiple processes share the same library in memory.

Advantages

- Smaller executable.
- Shared libraries reduce RAM usage.
- Easier maintenance.
- Libraries can be updated independently.

Disadvantages

- Increased startup time due to library loading.
 - Runtime symbol resolution overhead.
-

Comparison

Feature	Static Linking	Dynamic Linking
Compile Time	High	Low
Load Time	Fast	Slightly Slow
Execution Time	Fast	Slightly Slower
Memory Usage	High	Low
Library Sharing	Not Possible	Possible
Updates	Recompile Required	Easy

Recommended Loading Strategy

Use Dynamic Linking + Lazy Loading (On-demand Loading).

Reason

Instead of loading all 120 libraries during startup,

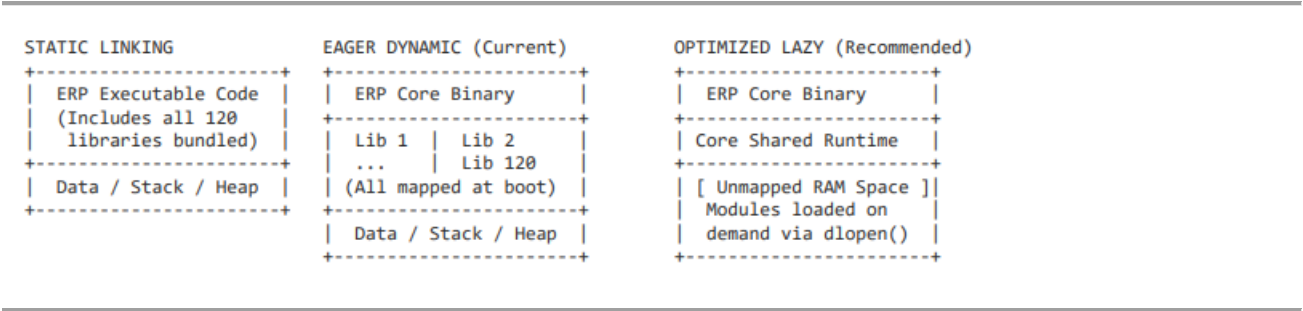
- Load only essential libraries.
- Load remaining libraries when first required.
- Frequently used libraries remain in memory.
- Rare libraries load only when called.

This significantly reduces startup delay while keeping memory utilization efficient.

Memory Diagram

Static Linking

Executable



Final Recommendation

Dynamic linking with lazy loading is the best solution because it

- reduces executable size,
- shares libraries among processes,
- minimizes memory usage,
- improves maintainability,
- decreases startup delay by loading libraries only when needed.

2. A cloud server simultaneously executes AI applications, database services, and web applications. Although 128 GB RAM is installed, memory allocation failures occur frequently. Investigate the causes by analyzing dynamic storage management, external fragmentation, and internal fragmentation. Design a memory management strategy that minimizes memory wastage while maintaining high throughput.

Problem

Cloud server specifications:

- 128 GB RAM
- AI Applications
- Database Services
- Web Applications

Still, memory allocation failures occur.

Possible Causes

1. Dynamic Storage Management

Memory is allocated and released continuously.

Example:

AI Process → Allocate 8 GB

Database → Allocate 12 GB

Web Server → Allocate 2 GB

AI Ends → Frees 8 GB

After repeated allocation/deallocation, memory becomes fragmented.

2. External Fragmentation

Free memory exists but is divided into many small blocks.

Example

Free 2GB

Used 4GB

Free 3GB

Used 6GB

Free 5GB

Need 8GB

Cannot allocate although

$2+3+5=10\text{GB}$

is available.

3. Internal Fragmentation

Occurs when allocated memory exceeds requested memory.

Example

Process requires

6 KB

Memory block

8 KB

Waste

2 KB

Memory Management Strategy

(a) Paging

Divide memory into fixed-size pages.

Advantages

- Eliminates external fragmentation.
 - Better utilization.
-

(b) Buddy Allocation

Split memory only as needed.

Advantages

- Fast allocation.
 - Easy merging.
 - Low fragmentation.
-

(c) Slab Allocation

Used for kernel objects.

Advantages

- Reduces allocation overhead.
 - High performance.
-

(d) Garbage Collection / Memory Pooling

Reuse released objects instead of repeated allocation.

Recommendation

Use

- Paging
- Buddy Allocation
- Slab Allocation
- Memory Pooling

Benefits

- Eliminates external fragmentation.
 - Reduces internal fragmentation.
 - Improves throughput.
 - Efficient utilization of 128 GB RAM.
-

3. **An airline reservation system experiences severe delays whenever thousands of customers book tickets simultaneously. The operating system frequently swaps processes between RAM and disk. Analyze how swapping affects CPU utilization and response time. Propose modifications to reduce swapping overhead without upgrading hardware.**

Problem

Thousands of users book tickets simultaneously.
Operating system continuously swaps processes.

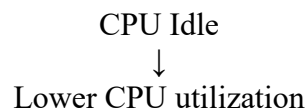
Swapping

Swapping moves processes
RAM $\rightleftharpoons$ Disk
when memory becomes insufficient.

Effects

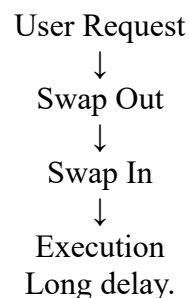
CPU Utilization

CPU waits while processes are moved.



Response Time

Each booking request waits for swapping.



Thrashing

Too much swapping results in
CPU spending more time swapping than executing.

Solutions

1. Increase Degree of Multiprogramming Control

Limit simultaneously running processes.

2. Use Demand Paging

Load only required pages.

3. Working Set Model

Keep only actively used pages in RAM.

4. Page Replacement

Use

- LRU

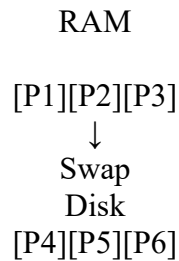
- Clock Algorithm
- to reduce unnecessary swapping.

5. Memory Compression

Compress inactive pages before swapping.

Diagram

Before



Continuous movement.

After Demand Paging

RAM

Only Active Pages

P1
P2
P5

Disk

Remaining Pages

Recommendation

Use

- Demand Paging
- Working Set
- LRU Replacement
- Controlled Multiprogramming

Benefits

- Higher CPU utilization
- Reduced response time
- Less disk I/O
- Improved throughput

4. A hospital management system allocates memory using contiguous memory allocation. After months of continuous operation, several large applications fail to load despite sufficient free memory. Investigate the reason using fragmentation concepts. Compare First Fit, Best Fit, Worst Fit, and Next Fit strategies, and recommend the most appropriate allocation technique for this scenario.

Problem

Applications fail although sufficient free memory exists.

Reason

External fragmentation.

Cannot allocate because memory is scattered.

Comparison

First Fit

Allocates first suitable block.

Advantages

- Fast.
- Less search time.

Disadvantages

- Creates fragmentation near beginning.
-

Best Fit

Allocates smallest sufficient block.

Advantages

- Less immediate waste.

Disadvantages

- Creates many tiny unusable holes.
 - Slow search.
-

Worst Fit

Uses largest block.

Advantages

- Leaves large remaining blocks.

Disadvantages

- Often wastes large memory regions.
-

Next Fit

Starts searching from previous allocation.

Advantages

- Faster than Best Fit.
- More balanced than First Fit.

Disadvantages

- Still causes fragmentation.
-

Comparison Table

Method	Speed	Fragmentation	Search Time
First Fit	Fast	Moderate	Low
Best Fit	Slow	High	High
Worst Fit	Medium	Moderate	Medium
Next Fit	Fast	Moderate	Low

Recommendation

Use **First Fit** with periodic **memory compaction**.

Reason

- Fast allocation.
 - Suitable for continuously running hospital systems.
 - Compaction removes fragmented free space.
-

Diagram

Before Compaction

2MB Free

Used

3MB Free

Used

5MB Free

After Compaction

Used

Used

Used

10MB Free

Large applications load successfully.

5. A banking application initially uses contiguous memory allocation but is later migrated to paging. Compare the performance of both approaches under heavy transaction loads. Evaluate page table overhead, fragmentation, memory utilization, and execution efficiency before recommending the migration strategy.

Problem

Banking application migrates from contiguous allocation to paging.
Need comparison.

Contiguous Allocation

Entire process occupies one continuous block.

Advantages

- Simple implementation.
- Fast address translation.

Disadvantages

- External fragmentation.
- Difficult expansion.
- Poor utilization.

Paging

Memory divided into fixed pages.

Process pages can occupy any frame.

Advantages

- No external fragmentation.
- Better utilization.
- Supports virtual memory.

Disadvantages

- Page table overhead.
- Small internal fragmentation.

Comparison

Feature	Contiguous Allocation	Paging
External Fragmentation	High	None
Internal Fragmentation	None	Small
Memory Utilization	Poor	High
Page Table Overhead	None	Present
Process Expansion	Difficult	Easy
Execution Efficiency	Decreases under load	Better under heavy load
Virtual Memory	Not Supported	Supported

Paging

RAM Frames

Frame1 → P1

Frame2 → P3

Frame3 → P2

Frame4 → P4

Pages need not be contiguous.

Recommendation

For a banking application under heavy transaction loads, **paging is the preferred memory management technique** because:

- it eliminates external fragmentation,
- improves memory utilization,
- allows processes to grow without requiring contiguous memory,
- supports virtual memory,
- provides better overall performance and scalability despite the additional page table overhead.

6. A gaming company develops an open-world game with multiple independent modules such as graphics, AI, physics engine, and audio processing. Analyze how segmentation can improve memory organization compared to paging. Design an appropriate segment table and explain how logical addresses are translated into physical addresses.

Problem Analysis

A gaming company develops an open-world game with independent modules:

- Graphics Engine
- AI Engine
- Physics Engine
- Audio Engine

The task is to analyze how **segmentation** improves memory organization compared to **paging**, design a **segment table**, and explain logical-to-physical address translation.

Segmentation

Segmentation divides a program into **logical units** based on functionality rather than fixed-size blocks.

Each module occupies a separate segment.

Example:

- Segment 0 → Graphics
- Segment 1 → AI
- Segment 2 → Physics
- Segment 3 → Audio

Each segment has:

- Base Address
 - Limit (Size)
-

Paging vs Segmentation

Feature	Paging	Segmentation
Division	Fixed-size pages	Variable-size logical modules
Programmer Visibility	Hidden	Visible
Fragmentation	Internal	External
Sharing	Difficult	Easy

Feature	Paging	Segmentation
Protection	Per page	Per module

Advantages of Segmentation

1. Better Memory Organization

Each module is stored independently.

Example

Graphics → Segment 0

AI → Segment 1

Physics → Segment 2

Audio → Segment 3

2. Easier Sharing

The graphics engine can be shared by multiple game levels.

3. Better Protection

Permissions can be assigned independently.

Example

Graphics → Read/Execute

AI → Read/Write

Physics → Read/Execute

Audio → Read Only

4. Independent Growth

The AI module can grow without affecting Graphics.

Segment Table

Segment No	Module	Base Address	Limit (KB)
0	Graphics	10000	500
1	AI	10500	300
2	Physics	10800	450

Segment No	Module	Base Address	Limit (KB)
------------	--------	--------------	------------

3	Audio	11250	250
---	-------	-------	-----

Logical Address

Logical Address consists of

<Segment Number, Offset>

Example

Logical Address

<2,150>

Meaning

Segment = Physics

Offset =150

Physical Address

Base + Offset

10800 +150

=10950

Address Translation Diagram

Logical Address

<Segment 2, Offset 150>

↓

Segment Table

Segment 2

Base =10800

Limit =450

↓

Physical Address

10800+150

=10950

Recommendation

Segmentation is the better choice because:

- Modules remain independent.
 - Easier debugging and maintenance.
 - Better protection.
 - Supports modular game development.
-

7. A software development company uses both segmentation and paging in its operating system. During debugging, developers notice increased address translation time. Analyze the advantages and disadvantages of segmentation with paging compared to pure paging and pure segmentation. Recommend an optimized architecture for the organization.

Problem Analysis

A software company uses **Segmentation with Paging**, but address translation has become slower.

Need comparison with:

- Pure Paging
- Pure Segmentation

Pure Paging

Memory divided into equal pages.

Advantages

- No external fragmentation.
- Easy allocation.

Disadvantages

- Internal fragmentation.
- No logical program structure.

Pure Segmentation

Memory divided into logical modules.

Advantages

- Easy sharing.
- Better protection.

Disadvantages

- External fragmentation.
- Difficult allocation.

Segmentation with Paging

Each segment is divided into pages.

Example

Graphics Segment



Page1

Page2

Page3



Frames

Advantages

Eliminates External Fragmentation

Paging removes holes.

Logical Organization

Each segment represents one module.

Better Protection

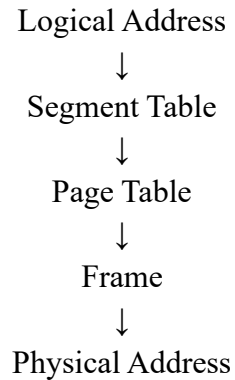
Protection at segment level.

Efficient Sharing

Entire segments can be shared.

Disadvantages

Double Address Translation



Extra lookup increases delay.

Larger Memory Overhead

Need both

- Segment Table
- Page Tables

Comparison

Feature	Paging	Segmentation	Segmentation + Paging
External Fragmentation	No	Yes	No
Internal Fragmentation	Yes	No	Small
Protection	Moderate	High	High
Sharing	Moderate	Excellent	Excellent
Translation Speed	Fast	Medium	Slow
Memory Overhead	Low	Medium	High

Optimization

Use

- Translation Lookaside Buffer (TLB)
- Multi-level page tables
- Frequently accessed segments cached in TLB

This significantly reduces translation delay.

Recommendation

Use **Segmentation with Paging + TLB** because it provides:

- Efficient memory utilization.
- Strong protection.

- Logical organization.
 - Fast address translation after caching.
-

8. A mobile operating system allows users to run multiple applications simultaneously, exceeding the available physical memory. Analyze how virtual memory enables efficient multitasking. Explain the consequences if virtual memory is disabled, and propose improvements for better memory performance.

Problem Analysis

Multiple mobile applications execute simultaneously even though RAM is limited.

Need to explain

- Virtual Memory
- Consequences if disabled
- Improvements

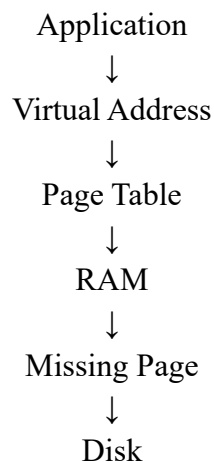
Virtual Memory

Virtual memory allows execution of programs larger than available RAM.

Inactive pages remain on disk.

Only required pages stay in RAM.

Working



Advantages

More Applications

Many apps execute simultaneously.

Better RAM Utilization

Only active pages occupy RAM.

Large Applications Supported

Games

Editors

Browsers

run even when RAM is limited.

Process Isolation

Each application gets its own virtual address space.

Improves security.

If Virtual Memory is Disabled

Fewer Applications

Limited by physical RAM.

Frequent Crashes

Large applications cannot load.

Poor Multitasking

Applications terminate frequently.

Lower User Experience

Background apps close automatically.

Improvements

Demand Paging

Load only required pages.

LRU Replacement

Remove least recently used pages.

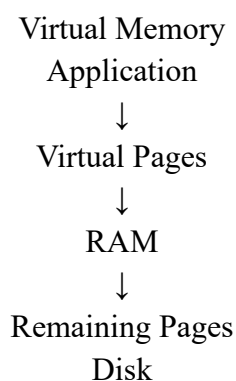
Memory Compression

Compress inactive pages.

Background Process Optimization

Suspend unused applications.

Diagram



Recommendation

Virtual memory should always remain enabled because it

- supports multitasking,
- increases memory efficiency,
- improves application performance,
- allows execution of large applications.

9. An online video editing application loads only required portions of videos into memory. Users report occasional delays when editing large files. Analyze how demand paging influences system performance. Suggest methods to reduce page faults without increasing RAM capacity.

Problem Analysis

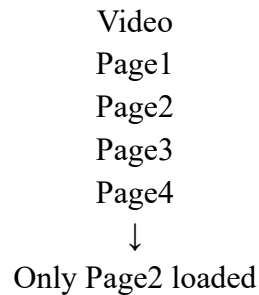
Only required portions of videos are loaded.

Large videos still experience editing delays.

Demand Paging

Pages are loaded only when first accessed.

Example



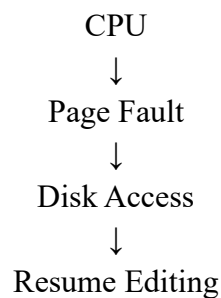
Advantages

- Lower memory usage.
- Faster initial loading.
- Better multitasking.

Causes of Delay

Page Fault

Required page not in RAM.



Disk access is much slower than RAM.

Large Video Files

Editing jumps between distant frames.

Many page faults occur.

Methods to Reduce Page Faults

Read-Ahead (Prefetching)

Load nearby pages before needed.

Increase Page Size (Moderately)

Reduces page table entries.

Better Replacement Algorithm

Use LRU instead of FIFO.

Working Set Model

Keep actively edited video frames in memory.

Cache Frequently Used Pages

Timeline thumbnails

Current editing frames

Audio cache

Diagram

Without Prefetching

Frame1



Disk



Delay



Frame2

With Prefetching

Frame1



Frame2



Frame3

Already Loaded

Recommendation

Use

- Demand Paging
- Read-ahead
- LRU Replacement
- Working Set Model

This minimizes page faults without increasing RAM.

10. A data analytics server experiences frequent page faults while processing large datasets. After monitoring memory references, the administrator notices repetitive loading and replacement of the same pages. Analyze the effectiveness of FIFO, LRU, Optimal, and Clock page replacement algorithms. Recommend the most suitable algorithm for this workload with proper justification.

Problem Analysis

Large datasets cause frequent page faults.

Same pages are repeatedly replaced.

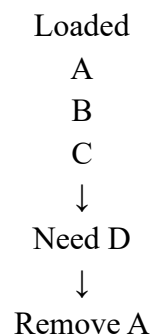
Need comparison of

- FIFO
- LRU
- Optimal
- Clock

FIFO (First In First Out)

Oldest page removed.

Example



Advantages

Simple.

Low overhead.

Disadvantages

May remove frequently used pages.

Can suffer from **Belady's Anomaly**, where increasing the number of frames may unexpectedly increase page faults.

LRU (Least Recently Used)

Removes least recently accessed page.

Advantages

- Good approximation of actual usage.
- High hit ratio.
- Suitable for repeated accesses.

Disadvantages

Needs tracking of page usage.

Optimal Algorithm

Removes the page that will not be used for the longest time in the future.

Advantages

- Minimum possible page faults.

- Benchmark for comparison.

Disadvantages

Impossible to implement in real systems because future memory references are unknown.

Clock Algorithm

Uses reference bits.

Recently used pages receive another chance.

Advantages

- Faster than LRU.
- Lower overhead.
- Good practical performance.

Disadvantages

Less accurate than LRU.

Comparison

Algorithm	Speed	Page Faults	Practical Use
FIFO	High	High	Basic systems
LRU	Medium	Low	Excellent
Optimal	Very Slow (Theoretical)	Minimum	Not practical
Clock	High	Low	Widely used

Workload Analysis

The server repeatedly accesses the same dataset.

This exhibits **temporal locality**, meaning recently used pages are likely to be used again.

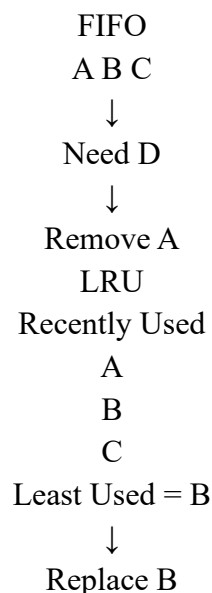
LRU performs exceptionally well because it keeps recently accessed pages in memory.

Clock is also efficient and requires less overhead.

FIFO performs poorly because it ignores usage history.

Optimal cannot be implemented.

Diagram



Clock
Reference Bit
A(1)
B(0)
C(1)
↓
Replace B

Recommendation

The most suitable algorithm is **LRU (Least Recently Used)** because:

- it exploits temporal locality,
- significantly reduces repeated page faults,
- improves execution efficiency for data analytics workloads.

If implementation overhead is a concern, the **Clock algorithm** is an excellent practical alternative, as it achieves performance close to LRU with much lower overhead.

11. A university examination server crashes during online examinations due to excessive page faults. Investigate whether thrashing is responsible for the failure. Analyze CPU utilization, page fault frequency, and working set size. Design an improved memory allocation policy to eliminate thrashing.

Problem Analysis

A university examination server crashes during online examinations because of **excessive page faults**. Thousands of students access the server simultaneously.

The objective is to determine whether **thrashing** caused the failure and propose an improved memory allocation policy.

What is Thrashing?

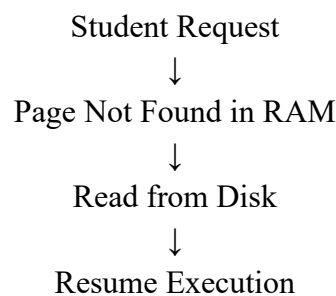
Thrashing is a condition where the operating system spends more time swapping pages between RAM and disk than executing processes. It occurs when the available physical memory is insufficient to hold the working sets of all active processes.

Indicators of Thrashing

1. High Page Fault Frequency (PFF)

Each page fault requires disk access, which is much slower than RAM.

Example:



A very high page fault rate indicates insufficient memory for the current workload.

2. CPU Utilization

During thrashing:

- CPU waits for page transfers.
- CPU utilization drops significantly.

Low CPU utilization with high disk activity is a strong symptom of thrashing.

3. Working Set Size

Each process requires a set of actively used pages (working set).

If:

$\text{Total Working Sets} > \text{Available RAM}$

the operating system continuously replaces pages, causing thrashing.

Cause of Server Failure

During examinations:

- Thousands of browser sessions start simultaneously.
- Authentication modules load.
- Database pages are accessed.
- Question papers and answer scripts are loaded.

The combined working sets exceed available RAM, leading to:

- Frequent page faults
 - Continuous swapping
 - Low CPU utilization
 - Server slowdown and eventual crash
-

Improved Memory Allocation Policy

1. Working Set Model

Allocate enough frames for each process based on its current working set.

Advantages:

- Keeps frequently used pages in RAM.
 - Reduces unnecessary replacements.
-

2. Page Fault Frequency (PFF) Control

Monitor page fault rate.

- High PFF → Allocate more frames.
 - Low PFF → Release unused frames.
-

3. Limit Degree of Multiprogramming

Restrict the number of simultaneously active examination sessions.

Waiting users remain in a queue until sufficient memory becomes available.

4. Priority-Based Allocation

Higher priority:

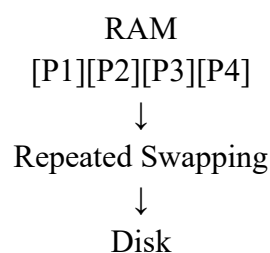
- Examination server
- Database

Lower priority:

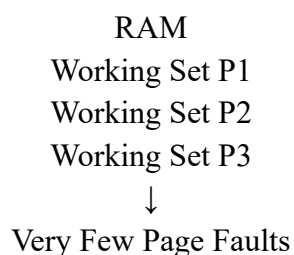
- Background logging
 - Analytics
-

Diagram

Before



After



Recommendation

The crash is most likely caused by **thrashing**. Implementing:

- Working Set Model
 - Page Fault Frequency Control
 - Controlled Multiprogramming
 - Priority-Based Memory Allocation
- will significantly improve system stability during online examinations.

12. A cloud provider hosts hundreds of virtual machines on a single physical server. During peak hours, several VMs become unresponsive. Analyze the role of working sets in preventing thrashing. Propose a memory allocation framework that dynamically adjusts working set sizes based on workload patterns.

Problem Analysis

A cloud provider hosts hundreds of virtual machines (VMs) on a single physical server.

During peak hours:

- Several VMs become slow.
 - Some become unresponsive.
- Need to analyze how working sets prevent thrashing.

Working Set

The working set is the collection of pages actively used by a process within a recent time interval.

Example:

VM1
Pages
2
5
6
9
11

These pages remain in RAM.

Importance

When every VM receives enough memory for its working set:

- Few page faults
- High CPU utilization
- Better throughput

Without sufficient working set:

VM
↓
Page Fault
↓
Disk Access
↓
Page Replacement
↓
Another Page Fault

Continuous swapping leads to thrashing.

Memory Allocation Framework

Step 1

Monitor page fault rate of every VM.

Step 2

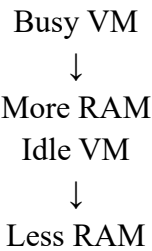
Calculate working set size.

Example

VM	Working Set
VM1	4 GB
VM2	6 GB
VM3	2 GB

Step 3

Allocate RAM dynamically.



Step 4

Balloon Driver

Reclaim unused memory from idle VMs.

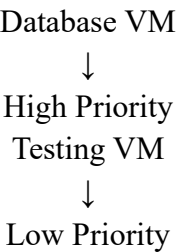
Allocate reclaimed memory to active VMs.

Step 5

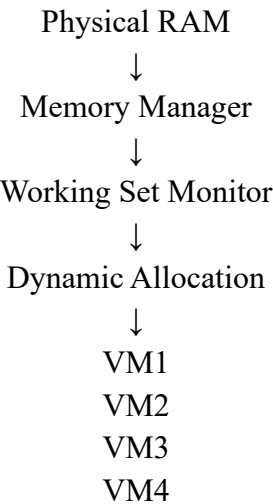
Priority Scheduling

Critical VMs receive memory first.

Example



Framework Diagram



Recommendation

Implement:

- Working Set Model
- Dynamic Memory Allocation
- Ballooning
- PFF Monitoring

This minimizes thrashing while improving responsiveness during peak workloads.

13. A multimedia company stores high-definition video files on HDDs while frequently accessed metadata is stored on SSDs. Analyze how disk devices influence virtual memory performance, swapping speed, and page replacement efficiency. Recommend a storage hierarchy that minimizes access latency.

Problem Analysis

The company stores:

- HD videos → HDD
- Metadata → SSD

Need to analyze disk influence on virtual memory.

HDD Characteristics

Advantages

- Large storage
- Low cost

Disadvantages

- Mechanical movement
- High seek time
- Slow swapping

SSD Characteristics

Advantages

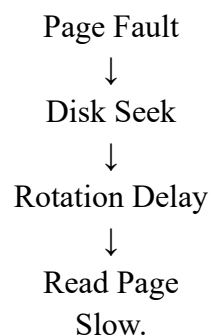
- No moving parts
- Very low latency
- Fast page loading

Disadvantages

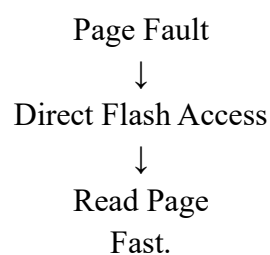
- Higher cost
- Limited write endurance

Virtual Memory Performance

HDD



SSD



Swapping Speed

HDD
RAM

↓
HDD
Slow
SSD
RAM
↓
SSD
Fast

Page Replacement Efficiency

Faster storage reduces the penalty of page replacement because swapped-out and swapped-in pages are transferred much more quickly.

Recommended Storage Hierarchy

CPU Cache
↓
RAM
↓
SSD
↓
HDD

Usage

RAM

- Active applications
- Swap space
- Frequently accessed metadata
- Frequently accessed pages

HDD

- Large video files
 - Backup
 - Archive
-

Recommendation

Store:

- Swap partition on SSD
 - Metadata on SSD
 - Videos on HDD
- This minimizes latency while reducing storage costs.

14. A software company plans to redesign its operating system memory manager. The proposed system integrates dynamic linking, demand paging, segmentation, virtual memory, and SSD-based swapping. Evaluate whether this integrated approach improves overall system performance. Identify possible bottlenecks and propose architectural improvements.

Problem Analysis

The proposed operating system combines:

- Dynamic Linking
 - Demand Paging
 - Segmentation
 - Virtual Memory
 - SSD Swapping
- Need to evaluate performance.

Component Analysis

Dynamic Linking

Advantages

- Shared libraries
 - Lower memory usage
- Disadvantage
- Startup overhead

Demand Paging

Advantages

- Load only required pages
 - Reduced RAM usage
- Disadvantage
- Initial page faults

Segmentation

Advantages

- Logical organization
 - Better protection
- Disadvantage
- External fragmentation (handled by paging)

Virtual Memory

Advantages

- Supports large applications
 - Better multitasking
- Disadvantage
- Thrashing if memory becomes insufficient

SSD Swapping

Advantages

- Fast swap operations
 - Lower page fault penalty
-

Overall Performance

Benefits

- Better memory utilization
 - Smaller executables
 - Faster application loading
 - High multitasking capability
 - Reduced swap delay
-

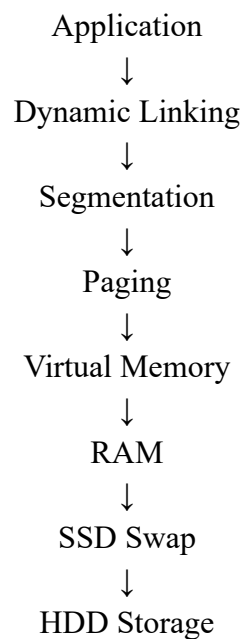
Possible Bottlenecks

- Excessive page faults
 - TLB misses
 - Dynamic library lookup overhead
 - SSD write wear
 - Large page tables
-

Architectural Improvements

- Multi-Level Page Tables
 - TLB
 - Working Set Model
 - Read-Ahead Prefetching
 - Frequently used libraries kept permanently in memory
 - Hybrid RAM + SSD caching
-

Architecture Diagram



Recommendation

The integrated approach provides the best balance of:

- Performance
- Memory efficiency
- Scalability

provided that advanced techniques like TLB caching, working set management, and SSD-based swapping are incorporated.

15. An e-commerce platform experiences inconsistent response times during festival sales. System logs reveal excessive dynamic library loading, frequent page replacements, and swapping activity. Analyze the interaction among dynamic linking, demand paging, and swapping. Develop a comprehensive optimization strategy that improves performance while minimizing memory consumption.

Problem Analysis

During festival sales:

- Slow response times
- Dynamic library loading
- Frequent page replacements
- Continuous swapping

Need to analyze their interaction and optimize performance.

Interaction of Components

Dynamic Linking

Loads shared libraries only when needed.

Benefits

- Smaller executables
- Shared memory usage

Issue

Repeated loading of commonly used libraries during peak traffic increases startup latency.

Demand Paging

Loads only required pages.

Benefits

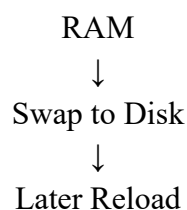
Efficient memory utilization

Issue

High request rates generate many page faults.

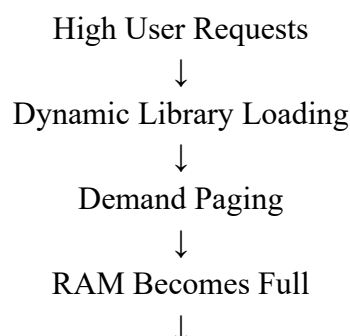
Swapping

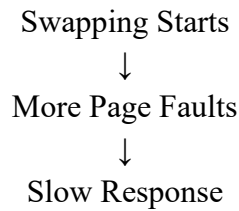
When RAM becomes full:



Excessive swapping increases response time and reduces CPU utilization.

Combined Effect





Optimization Strategy

1. Preload Frequently Used Libraries

Keep payment, authentication, and product catalog libraries permanently in memory.

2. Demand Paging with Read-Ahead

Prefetch pages likely to be accessed next.

3. LRU or Clock Page Replacement

Avoid replacing recently used pages.

4. Working Set Allocation

Allocate enough frames to active services.

5. SSD-Based Swap Space

Reduce swap latency significantly compared to HDD.

6. Increase Shared Library Reuse

Allow multiple application processes to share a single copy of commonly used dynamic libraries.

7. Reduce Degree of Multiprogramming

Limit excessive background processes during peak sales.

Optimized Architecture

Clients



Load Balancer



Application Server



Shared Dynamic Libraries



Demand Paging



RAM



SSD Swap



Database

Recommendation

To achieve consistent performance during festival sales:

- preload frequently used libraries,

- use demand paging with read-ahead,
- adopt LRU or Clock page replacement,
- manage memory using the Working Set Model,
- place swap space on SSD,
- maximize shared library reuse.

16. A research laboratory develops a scientific simulation requiring large matrices exceeding available RAM. Compare the effectiveness of virtual memory, segmentation, paging, and segmentation with paging. Recommend the most efficient memory management model by considering scalability, performance, and memory protection.

Problem Analysis

A research laboratory develops scientific simulations involving **very large matrices** (e.g., weather modeling, molecular simulations, AI training) that exceed the available physical RAM.

The objective is to compare:

- Virtual Memory
- Paging
- Segmentation
- Segmentation with Paging

and recommend the most efficient memory management model.

1. Virtual Memory

Virtual memory allows programs larger than RAM to execute by storing inactive pages on secondary storage.

Advantages

- Supports programs larger than physical RAM.
- Improves multitasking.
- Efficient RAM utilization.

Disadvantages

- Page faults increase execution time.
- Excessive swapping may cause thrashing.

2. Paging

Memory is divided into fixed-size pages and frames.

Advantages

- Eliminates external fragmentation.
- Efficient memory allocation.
- Easy page replacement.

Disadvantages

- Internal fragmentation.
- Large page table overhead.

3. Segmentation

Program divided into logical modules.

Example

Segment 0 → Matrix Storage

Segment 1 → Mathematical Functions

Segment 2 → Visualization

Segment 3 → Input/Output

Advantages

- Logical organization.
- Easy protection.
- Better module sharing.

Disadvantages

- External fragmentation.

4. Segmentation with Paging

Each logical segment is divided into pages.

Advantages

- Logical organization.
- Eliminates external fragmentation.
- Supports protection and sharing.

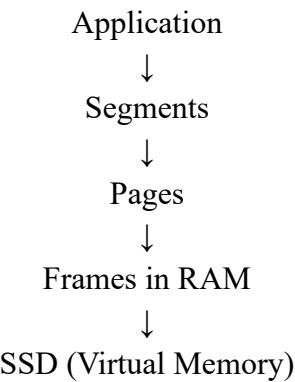
Disadvantages

- Double address translation.
- More memory overhead.

Comparison

Feature	Virtual Memory	Paging	Segmentation	Segmentation + Paging
Scalability	Excellent	Good	Moderate	Excellent
External Fragmentation	No	No	Yes	No
Internal Fragmentation	Small	Yes	No	Small
Protection	Good	Good	Excellent	Excellent
Memory Utilization	High	High	Moderate	Very High
Performance	Good	Good	Moderate	Very Good

Diagram



Recommendation

The best choice is **Segmentation with Paging combined with Virtual Memory** because it:

- supports extremely large datasets,
- provides logical organization,
- eliminates external fragmentation,
- offers strong memory protection,
- scales efficiently for scientific applications.

- 17. A cybersecurity organization requires strict memory protection between user processes and kernel processes. Analyze how segmentation, paging, and segmentation with paging provide isolation and protection. Design a secure memory architecture capable of preventing unauthorized memory access while maintaining execution efficiency.**

Problem Analysis

The organization requires strict isolation between:

- User Processes
- Kernel Processes

to prevent unauthorized memory access.

1. Segmentation

Each process is divided into logical segments.

Example

Code Segment

Data Segment

Stack Segment

Protection bits:

- Read
- Write
- Execute

Advantages

- Fine-grained protection.
- Module isolation.

Disadvantage

- External fragmentation.
-

2. Paging

Memory divided into pages.

Each page has protection bits.

Example

Page

Read Only

Read/Write

Kernel Only

Advantages

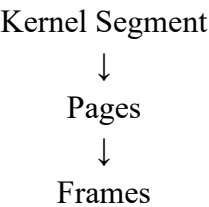
- No external fragmentation.
 - Fast allocation.
-

3. Segmentation with Paging

Each segment contains multiple pages.

Protection exists at both levels.

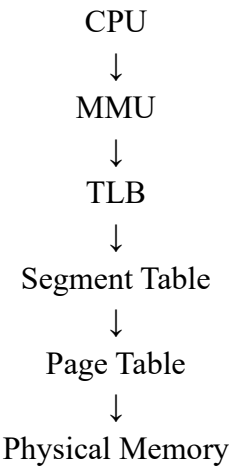
Example



Benefits

- Logical protection.
- Efficient allocation.
- Strong isolation.

Secure Memory Architecture



Memory Layout



Security Features

- Supervisor mode protection
 - Read/Write/Execute permissions
 - Address Space Layout Randomization (ASLR)
 - NX (No Execute) pages
 - TLB protection
 - Process isolation
-

Recommendation

Use **Segmentation with Paging** because it combines:

- logical isolation,
- efficient allocation,
- strong access control,
- better scalability than pure segmentation.

18. A fintech company is designing a next-generation operating system for financial servers. The system must support secure memory allocation, efficient page replacement, fast loading of shared libraries, and minimal disk I/O. Develop a complete memory management framework integrating dynamic storage management, paging, virtual memory, working sets, and SSD storage. Justify each design decision.

Problem Analysis

The financial server requires:

- Secure memory allocation
 - Efficient page replacement
 - Fast shared library loading
 - Minimal disk I/O
-

Proposed Framework

Dynamic Storage Management

Use **Buddy Allocation**

Advantages

- Fast allocation
 - Easy merging
 - Low fragmentation
-

Paging

Fixed-size pages.

Benefits

- Eliminates external fragmentation.
-

Virtual Memory

Supports applications larger than RAM.

Working Set Model

Keep frequently accessed financial data in RAM.

Page Replacement

Use **LRU**.

Reason

Financial applications repeatedly access recent transactions.

Dynamic Linking

Share common libraries:

- Encryption
- Authentication
- Database APIs

Benefits

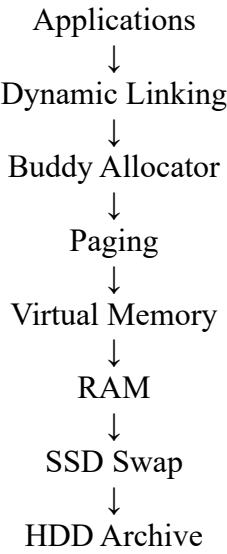
- Lower memory usage.
 - Faster updates.
-

SSD Storage

Store:

- Swap Space
- Frequently accessed logs
- Metadata

Framework Diagram



Justification

Component	Reason
Buddy Allocation	Low fragmentation
Paging	Efficient allocation
Virtual Memory	Large applications
LRU	Few page faults
Working Set	Prevents thrashing
SSD	Fast swap
Dynamic Linking	Shared libraries

Recommendation

This integrated framework provides:

- High security
- Low memory wastage
- Fast execution
- Reduced disk I/O

making it ideal for financial servers.

19. An autonomous vehicle runs perception, navigation, and object detection modules concurrently on limited embedded memory. Analyze how demand paging, page replacement policies, virtual memory, and working set models influence real-time performance. Recommend a customized memory management strategy that guarantees deterministic execution with minimal latency.

Problem Analysis

Modules:

- Perception
- Navigation
- Object Detection

execute simultaneously on limited embedded memory.

The system must guarantee **deterministic execution**.

Demand Paging

Advantages

- Low RAM usage.

Disadvantages

- Page faults introduce unpredictable delays.

Not suitable for safety-critical tasks.

Virtual Memory

Provides large address space.

However,

excessive paging may violate real-time deadlines.

Page Replacement

FIFO

Simple but poor performance.

LRU

Good performance but tracking overhead.

Clock

Efficient compromise.

Working Set

Maintain active pages for:

- Navigation
- Sensors
- Camera
- Object Detection

Avoid swapping these pages.

Customized Strategy

Critical Modules

Lock pages permanently in RAM.

Example

- ✓ Navigation
- ✓ Object Detection

✓ Emergency Braking
Never swapped.

Non-Critical Modules

Use demand paging.

Example

- ✓ Media
 - ✓ Logging
 - ✓ Diagnostics
-

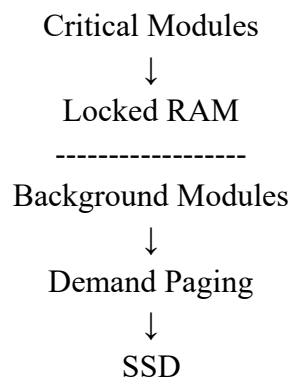
Page Replacement

Clock algorithm for non-critical pages.

SSD Swap

Use only for background applications.

Diagram



Recommendation

For autonomous vehicles:

- Lock critical pages in RAM.
 - Use demand paging only for non-critical tasks.
 - Use the Clock algorithm.
 - Maintain working sets for real-time modules.
- This guarantees deterministic execution with minimal latency.

20. A technology company plans to build an operating system for future AI-powered supercomputers. The system must efficiently support millions of concurrent processes, shared libraries, heterogeneous memory devices, SSDs, and distributed storage. Design a complete memory management architecture integrating linkers, dynamic linking, dynamic storage management, swapping, contiguous allocation, paging, segmentation, segmentation with paging, virtual memory, demand paging, page replacement, thrashing prevention, working sets, and advanced disk management. Evaluate trade-offs among performance, scalability, reliability, and cost.

Problem Analysis

The operating system must support:

- Millions of processes
 - Shared libraries
 - SSDs
 - Distributed storage
 - AI workloads
-

Proposed Architecture

1. Dynamic Linking

Shared AI libraries:

- Tensor operations
- Neural network kernels
- CUDA/OpenCL runtimes

Benefits

- Lower memory usage.
 - Faster updates.
-

2. Dynamic Storage Management

- Buddy Allocation
- Slab Allocation

for kernel objects.

3. Paging

Used for all user processes.

4. Segmentation

Separate:

- Code
 - Data
 - Stack
 - Shared Libraries
-

5. Segmentation with Paging

Each segment divided into pages.

Benefits

- Logical organization.
 - Efficient allocation.
-

6. Virtual Memory

Supports applications larger than RAM.

7. Demand Paging

Load only required AI model pages.

8. Page Replacement

Clock-Pro or LRU (depending on implementation complexity)

Keeps recently used pages.

9. Working Set Model

Monitor every process.

Allocate RAM according to active working sets.

10. Thrashing Prevention

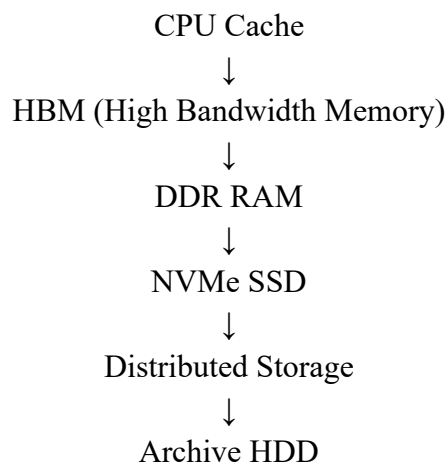
- Working Set Model
 - Page Fault Frequency (PFF)
 - Dynamic frame allocation
 - Controlled multiprogramming
-

11. Swapping

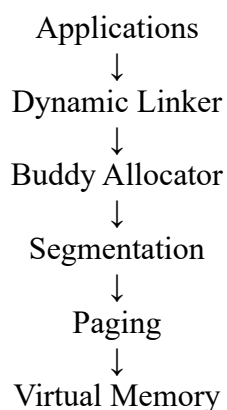
Use SSD-based swapping.

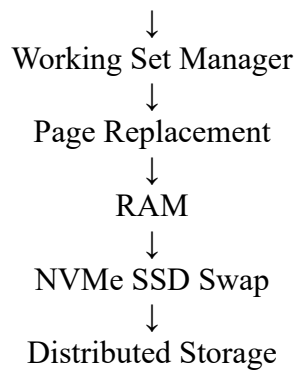
Cold data moves to distributed storage when appropriate.

12. Disk Hierarchy



Complete Architecture





Trade-Off Analysis

Feature	Advantages	Disadvantages
Dynamic Linking	Shared libraries	Startup overhead
Paging	Eliminates external fragmentation	Page table overhead
Segmentation	Logical protection	Complex management
Virtual Memory	Supports huge applications	Page faults
Demand Paging	Efficient RAM usage	Initial delays
SSD Swapping	Fast disk access	SSD wear over time
Working Set	Prevents thrashing	Monitoring overhead
Buddy Allocation	Fast allocation	Some internal fragmentation

Final Recommendation

A **hybrid memory management architecture** integrating:

- Dynamic Linking
 - Buddy Allocation
 - Segmentation with Paging
 - Virtual Memory
 - Demand Paging
 - LRU/Clock-based Page Replacement
 - Working Set Model
 - SSD/NVMe-based Swapping
 - Multi-level memory hierarchy (HBM → RAM → SSD → Distributed Storage)
- is the most suitable solution for future AI-powered supercomputers.